UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE ESTUDIOS SUPERIORES ARAGÓN

EXAMEN PRÁCTICO

Compresión de imágenes mediante Quadtrees
Estrategia Divide y Vencerás

DISEÑO Y ANÁLISIS DE ALGORITMOS
Grupo 1510 · 2027-I

Integrantes (orden alfabético)

Carlos Daniel Balanzar Velasco

Giner Ortiz Vicente

Lara Hernandez Elias Lemuel

Martinez Guillen Martin

Profesor: Víctor Manuel Sánchez Sánchez

9 de octubre de 2026

A. Modelado del problema

El objetivo es construir una representación jerárquica de imágenes RGB que aproveche las regiones de color similar. La entrada es una matriz de alto × ancho × 3 (valores enteros de 0 a 255) y una tolerancia no negativa τ. La salida es un árbol Quadtree, una reconstrucción RGB y métricas de costo, almacenamiento y fidelidad.

Representación y criterio de uniformidad

Cada nodo registra coordenadas (x, y), ancho, alto, un color RGB si es hoja y una lista de hijos si se subdivide. Para una región R de píxeles, se obtiene su media RGB μ por canal. Se considera uniforme cuando $\max |p_c - \mu_c| \leq \tau$, evaluando todos los píxeles y los tres canales. El color representativo se redondea al entero más cercano.

B. Diseño algorítmico

Dividir: se comprueba el criterio de uniformidad; si falla, se calculan las mitades enteras de ancho y alto y se forman hasta cuatro subregiones no vacías. Resolver: el algoritmo se llama recursivamente para cada subregión. Combinar: se crea un nodo padre con las referencias a sus hijos. La recursión se detiene al obtener una región uniforme o un único píxel.

Para reconstruir, se recorre el árbol: cada hoja pinta en la matriz de salida la región que representa. Con dimensiones impares, los segmentos tienen tamaños $\lfloor n/2 \rfloor$ y $\lceil n/2 \rceil$; se omiten divisiones vacías, por lo que no se pierden píxeles.

Pseudocódigo de la estrategia

Operación	Descripción
Construir(R, τ)	Calcular media y máxima desviación; si cumple tolerancia, devolver hoja con color medio; de otro modo subdividir y construir hijos.
Reconstruir(nodo)	Si es hoja, escribir su color en toda su región; si es interno, reconstruir cada hijo.

La implementación utiliza NumPy para cálculos sobre matrices; la definición del nodo, la división recursiva y la reconstrucción fueron programadas explícitamente en quadtree.py.

C. Correctitud y análisis de complejidad

Justificación de correctitud

Base: una hoja almacena la posición, las dimensiones y el color con el que debe reconstruirse su región. Paso inductivo: si una región no puede representarse como hoja, se divide en cuadrantes disjuntos cuya unión cubre la región original. Suponiendo correcta la reconstrucción de cada hijo, la del padre es correcta al unir sus regiones sin huecos ni superposiciones. Terminación: en cada división las dimensiones positivas disminuyen hasta alcanzar 1×1 o la tolerancia.

La corrección se refiere a reconstruir la representación aproximada definida por el árbol, no a preservar los colores originales con $\tau > 0$. Con $\tau = 0$, una hoja solo agrupa píxeles idénticos y la reconstrucción resulta exacta.

Costo temporal

Sea n el lado de una imagen cuadrada RGB, asumiendo n potencia de dos. Calcular la media y la máxima desviación de una región de lado n cuesta $\Theta(n^2)$ porque examina sus píxeles. Este costo no puede omitirse del análisis.

Escenario	Recurrencia y solución
Mejor caso: uniforme	$T(n) = \Theta(n^2)$, un solo nodo.
Peor caso: subdivisión completa	$T(n) = 4T(n/2) + \Theta(n^2) = \Theta(n^2 \log n)$, por el segundo caso del método maestro.
Reconstrucción	$O(n^2 + N)$, donde N es el número de nodos; se pintan en total n^2 píxeles.

Costo espacial

El árbol puede contener $\Theta(n^2)$ nodos en el peor caso. La pila recursiva tiene profundidad $O(\log n)$ para imágenes cuadradas, y las matrices de entrada y reconstrucción requieren $\Theta(n^2)$ bytes a un número constante de canales. Las expresiones temporales de NumPy también pueden necesitar memoria proporcional al área de la región examinada; el espacio total es $O(n^2)$.

Modelo de almacenamiento y calidad

Para comparar contra RGB sin comprimir se usa una codificación teórica por recorrido preorden: 64 bits para ancho/alto, 1 bit por nodo interno y 25 bits por hoja (indicador y color RGB de 24 bits). Las dimensiones de las regiones se deducen de su posición en el árbol. El tamaño total se redondea a bytes; no representa el consumo real de objetos Python ni el tamaño de JPG/PNG.

Razón = bytes RGB / bytes Quadtree. MSE = promedio de (píxel original − reconstruido)² sobre los tres canales. Razón mayor que 1 implica ahorro; menor que 1 implica expansión.

D. Resultados experimentales

Se generaron tres clases de imagen sintética: uniforme (un único color), regiones (cuatro cuadrantes de colores sólidos) y detalles (ruido RGB pseudoaleatorio con semilla 42). Se probaron tamaños 64^2, 128^2 y 256^2, con tolerancias 0, 20 y 60: 27 configuraciones. Cada construcción se ejecutó 5 veces con perf_counter, para un total de 135 construcciones; el CSV presenta media, mediana y desviación estándar del tiempo.

Comparación seleccionada: 256 × 256, tolerancia 20

Contenido	Tiempo medio	Nodos	Razón	MSE
Uniforme	2.00 ms	1	16,384:1	0
Regiones	3.06 ms	5	9,362.29:1	0
Detalles	686.73 ms	87,381	0.947:1	0

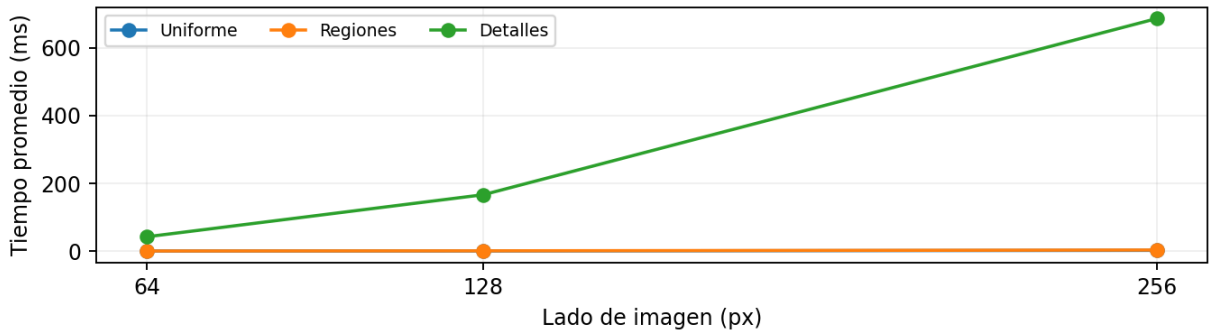


Figura 1. Tiempo promedio de construcción por tamaño (tolerancia 20). Fuente: experimentos.csv.

Interpretación del costo

Las imágenes uniformes necesitan una sola hoja y las de regiones definidas cinco nodos, incluso cuando crece la resolución. Aunque el número de nodos no aumenta, el tiempo sí lo hace porque la comprobación de uniformidad examina los píxeles. En contraste, el ruido obliga a descomponer casi toda la imagen: a 256^2 produce 87,381 nodos y tarda aproximadamente 0.687 s.

Los tiempos son mediciones de una ejecución concreta, no garantías de rendimiento en otros equipos. Las cinco repeticiones aportan una medida de variabilidad, pero para una comparación más rigurosa podrían intercalarse pruebas y repetirse en sesiones distintas.

D. Resultados experimentales (continuación)

Calidad frente a almacenamiento

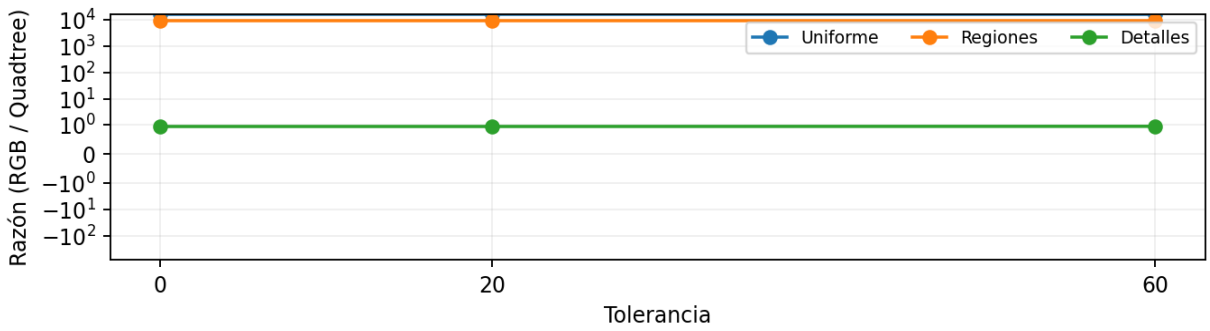


Figura 2. Razón de compresión a 256 × 256 para las tres tolerancias (escala simétrica-logarítmica).

A 256², las imágenes uniforme y de regiones son exactas (MSE = 0) en las tres tolerancias y conservan razones de 16,384:1 y 9,362.29:1. Para ruido, tolerancias 0 y 20 originan 87,381 nodos, razón 0.947:1 y MSE = 0: la representación ocupa cerca de 5.56% más que RGB sin comprimir.

Al elevar la tolerancia a 60 en la imagen de ruido de 256², los nodos descienden a 86,905 y la razón sube a 0.953:1; sin embargo, aparece distorsión (MSE = 7.3635). Es una mejora limitada de almacenamiento y sigue sin producir ahorro neto.

E. Conclusiones

Los Quadtrees resultan especialmente adecuados para imágenes con grandes áreas uniformes o bloques definidos. Su eficiencia depende de la estructura espacial y no únicamente del tamaño de entrada. La tolerancia ofrece un compromiso entre calidad y almacenamiento: aumentar su valor puede disminuir subdivisiones, pero introduce pérdida de detalle.

En imágenes de variación intensa, como ruido aleatorio, el árbol puede ser más grande que los datos RGB originales, además de exigir más tiempo. La elección del umbral debe atender simultáneamente al MSE, la razón de compresión y el tiempo de construcción. Como mejoras futuras se proponen probar fotografías naturales, guardar el árbol en un formato binario real y comparar otros criterios de uniformidad.

Alcance de las mediciones

Los tamaños Quadtree son estimaciones de una codificación definida para el análisis; no se ha implementado su serialización binaria. Los resultados experimentales corresponden a tres patrones sintéticos y no permiten generalizar el desempeño a todas las imágenes fotográficas.

Referencias y materiales de apoyo

Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2022). Introduction to Algorithms (4.ª ed.). MIT Press.

Samet, H. (1984). The quadtree and related hierarchical data structures. ACM Computing Surveys, 16(2), 187-260.

Documentación oficial de NumPy: numpy.org/doc.

Código fuente del proyecto: quadtree.py; experimentación: experimentos.py; resultados reproducibles: experimentos.csv (entregables del equipo).

Nota editorial: los acentos de los nombres propios en la portada se normalizaron según la grafía usual del español cuando correspondía.